

TECNOLOGÍAS DE LA INFORMACIÓN EN LÍNEA

DESARROLLO DE SISTEMAS INFORMÁTICOS

3 créditos

Profesor:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
DESARROLLO DE SISTEMAS INFORMÁTICOS	Quinto

Tutorías: El profesor asignado se publicará en el entorno virtual de aprendizaje (online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección

Presentación.

PERÍODO SEPTIEMBRE 2025 / ENERO 2026

Índice

Tabla de contenido

Unidad I Desarrollo de software	3
Tema 1: introducción	3
1.1 Definición de diseño de software	3
1.2 Principios de diseño de software	3
1.3 Importancia del diseño de software	11
1.4 Herramientas para la construcción de proyectos de software	12
1.5 Herramientas para el control de versiones de software	18
Tema 2: Paradigmas de diseño	23
2.1 Diseño estructurado	23
2.2 Diseño orientado a objetos.....	27
2.3 Diseño orientado a eventos.....	30
2.4 Diseño orientado al nivel de componentes	34
2.5 Diseño centrado en los datos.....	35



Organización de la lectura para el estudiante por semana del compendio

Semanas	Páginas
Semana 1	Página 3 - 15
Semana 2	Página 16 - 25
Semana 3	Página 26 – 39



Resultado de aprendizaje de la asignatura

Este curso provee a los estudiantes el conocimiento y la experiencia práctica para diseñar e implementar aplicaciones web, móvil o de escritorio cumpliendo con los estándares actuales y las buenas prácticas de programación que faciliten su mantenibilidad, escalabilidad y adaptabilidad.



Ilustraciones gráficas



Sabías que. - La presente imagen dentro del manual mostrara información interesante y novedosa de la asignatura.



Recuerde que. - La presente imagen dentro del manual permite recordar información que es relevante y que vas necesitar en tu vida profesional.



Comprueba tu aprendizaje. - Es un cuestionario de un conjunto de preguntas que se confecciona para obtener información con algún objetivo en concreto. Por cada tema de la unidad se tendrá cuestionario que el estudiante debe resolver entre preguntas teóricas y prácticas.



Videos. - Para complementar contenidos de la unidad dentro del manual se tiene videos que permitirá al estudiante revisar y explorar conocimientos auditivos y visuales.



Curiosidades. - La presente imagen en el manual mostrara información que debes conocer de la asignatura.



Datos útiles. - La presente imagen en el manual mostrara información que deberás tomar en cuenta en otras unidades de la asignatura o de otras asignaturas en semestre superiores.



DESARROLLO DE SISTEMAS INFORMÁTICOS



Unidad I Desarrollo de software

Resultado de aprendizaje de la unidad: Diseñar un producto de software en el que se apliquen principios de diseño, para que sea robusto, intuitivo, fácil de mantener y modificar.



Tema 1: introducción

1.1 Definición de diseño de software

Diseño de software es el proceso de diseño para la planificación de una solución de software. Este proceso es, por regla general, necesaria para que los programadores puedan manejar la complejidad que la mayoría de los programas informáticos poseen y para disminuir el riesgo de desarrollos erróneos.

1.2 Principios de diseño de software

Un principio se puede entender como una guía de comportamiento amplia aplicable a muchas situaciones. En general un principio no te dice que hacer exactamente, sino que te da pistas de cuál es la acción correcta a través de una gran cantidad de situaciones, pero los detalles están a cargo de ti mismo.

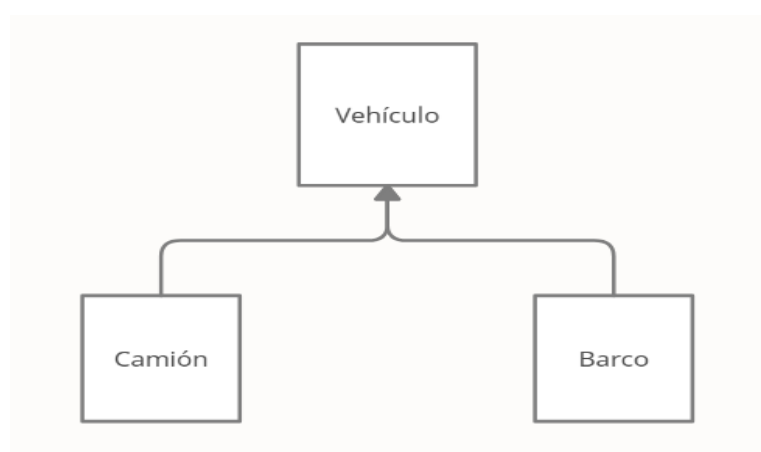
Hablando de principios de diseño de software, puedes pensar en ellos como en un faro que te guía a través de la oscuridad de los requerimientos del problema. A diferencia de los patrones de diseño, no establecen los pasos necesarios para aplicarlos, ni siquiera las situaciones en las que son aplicables, de hecho, se pueden crear varios patrones y reglas basándonos en ellos. Los principios más conocidos son los expuestos a continuación, es importante conocerlos, pero más aún entenderlos para así poder tomar la mejor decisión al momento de aplicarlos dependiendo de la situación o problemática a resolver (Software Design & Architecture map, 2021).

Composición sobre Herencia

La composición sobre la herencia (o principio de reutilización compuesta) en la programación orientada a objetos es una técnica mediante la cual las clases pueden lograr un comportamiento polimórfico y la reutilización del código al contener otras clases que implementan la funcionalidad deseada en lugar de a través de la herencia.

La herencia se utiliza en la programación orientada a objetos como una forma de reutilizar código a través de diferentes objetos. Podemos definir estructuras de objetos padre-hijo, por lo que los métodos y las variables se pasan de los padres a sus hijos: el código se hereda. Este es un mecanismo poderoso para evitar la replicación de código - ser DRY -, pero tiene sus inconvenientes (SOLID Principles, 2021).

Figura 1 Esquema de Herencia



El problema es que, en una estructura de padres e hijos, los hijos dependen de sus padres. Si el comportamiento de un padre cambia, todos los comportamientos de sus hijos cambiarán automáticamente. O si se agrega un nuevo comportamiento al padre, todos los hijos también heredarán ese comportamiento.

En el siguiente ejemplo, el usuario quiere crear una aplicación que trate con diferentes tipos de patos. Cada pato vuela, nada y grazna, por lo que para reutilizar esos comportamientos se crea un supertipo de pato con los métodos graznar, nadar y volar. Todos los patos del sistema heredarán de la clase Pato y, por lo tanto, también heredarán esos comportamientos.

El problema surge cuando se introduce en el sistema un nuevo tipo de pato que no debería volar: el PatoDeGoma. Si hacemos que este nuevo tipo lo herede de Pato, ¡haremos que el pato de goma vuele! Eso está mal. ¡Pero si no usamos la herencia tendremos que replicar los métodos de graznido y natación! Eso no es realmente DRY. Entonces, ¿qué debemos hacer?

La solución a esto es utilizar la composición para la reutilización del código en lugar de la herencia. Cada uno de los comportamientos que un pato podría necesitar (graznar, nadar, volar) se dividirá en una nueva clase: Graznido, Natación, Vuelo. Luego, cada tipo de pato diferente usará solo los comportamientos necesarios (SOLID Principles, 2021).

El código se verá así:

Figura 2. Reutilización de código

```
public interface ICapacidadVolar
{
    public void volar();
}

public class TieneCapacidadVolar: ICapacidadVolar
{
    public void volar()
    {
        // Flying code
    }
}

public class NoTieneCapacidadVolar: ICapacidadVolar
{
    public void volar()
    {
        // Do nothing
    }
}
```



Curiosidades. - El método de depuración del patito de goma es un término informal utilizado en ingeniería de software para describir un método de revisión de código en donde un programador toma un [patito de goma](#) y revisa su código forzándose a sí mismo a explicarlo, línea por línea, al pato

Figura 3. Código Patito de Goma

```
public abstract class Pato
{
    protected ICapacidadVolar capacidadVolar;
    public virtual void realizarVuelo()
    {
        this.capacidadVolar.volar();
    }
}

public class PatoReal
{
    public PatoReal()
    {
        capacidadVolar = new tieneCapacidadVolar();
    }
}

public class PatoDeGoma
{
    public PatoDeGoma()
    {
        capacidadVolar = new NoTieneCapacidadVolar();
    }
}
```

Encapsula lo que varía

Un diseño es mejor cuando las partes que varían se encapsulan en un módulo separado.

Este principio tiene dos aspectos que corresponden aproximadamente a los dos subprincipios SRP y OCP. El primero se trata de realizar cambios locales. Todo lo que se supone que cambiará en el futuro debe encapsularse en un solo módulo. Esto significa que se evitan en la medida de lo posible las preocupaciones transversales. Esto no es completamente posible, pero en muchos casos lo es.

El segundo aspecto trata sobre la introducción de abstracciones. En ocasiones, el concepto de variación es uno que varía en tiempo de ejecución en lugar de por mantenimiento. Entonces, en tiempo de ejecución, se decide sobre una cierta variación o incluso puede haber varias variaciones al mismo tiempo. En este caso, tiene que haber una clase base abstracta o una interfaz que encapsule el concepto variable. Luego, varias clases de descendientes concretos especifican la variación concreta. La diferencia entre los dos aspectos es si el concepto variable es uno que cambia con el tiempo durante el mantenimiento o uno que puede cambiar en tiempo de ejecución. Sin embargo, el consejo es el mismo: encapsular el concepto que varía (Encapsulate the Concept that Varies, 2021).

Razones

Hay dos razones para este principio. La primera razón es la localización. Cuando un concepto variable se encapsula correctamente en un solo módulo, solo este módulo se ve afectado en caso de un cambio. Esto reduce el esfuerzo de mantenimiento y el efecto dominó.

La segunda razón entra en juego cuando el concepto variable se implementa como una clase o interfaz abstracta. En este caso, se puede introducir una variación sin cambiar el código existente y probado. Esto reduce el esfuerzo de prueba (ya que el código ya probado no necesita volver a probarse ya que no se cambia) así como los efectos dominó (ya que la mejora se realiza simplemente agregando una clase. Tenga en cuenta que para que esta justificación funcione, el principio de sustitución de Liskov también debe cumplirse (Encapsulate the Concept that Varies, 2021).

Estrategias

Introducir un módulo separado para el concepto que puede cambiar en el futuro. De esa manera, el cambio futuro solo afectará a ese módulo en particular. Si el concepto variable está correctamente encapsulado, solo este módulo tendrá que cambiar.

Introducir una interfaz que encapsule el concepto variable. La interfaz puede implementarse de manera diferente mediante varias clases y el código que solo se basa en la interfaz puede manejar cualquier clase que implemente la interfaz. En caso de otra variación, solo se debe introducir otra clase y esta clase tiene que implementar la interfaz. Si la abstracción se realiza correctamente, ningún módulo tiene que cambiar.

Introducir una clase base abstracta que encapsule el concepto variable. Esto es básicamente lo mismo que introducir una interfaz. Pero aquí también se puede heredar la implementación. Por lo tanto, las partes comunes pueden permanecer en la clase base abstracta, mientras que solo las variaciones reales se definen en las subclases. Mediante la anulación de métodos, la implementación de los métodos de la clase base se puede cambiar sin tocar la clase base directamente (Importance of software design, 2021).

Usa patrones de diseño. Varios patrones de diseño utilizan las técnicas anteriores para encapsular diferentes conceptos. Por ejemplo:

- Abstract Factory: una familia de objetos cambia.
- Factory method: el tipo exacto de objeto para crear cambios.
- Adapter: la interfaz de un módulo cambia.
- Bridge: un concepto varía en más de un aspecto.
- Decorator: Es posible que sea necesario mejorar el comportamiento de ciertos métodos.
- Iterator: El algoritmo de recorrido de una estructura cambia. O la estructura en sí cambia, lo que resulta en la necesidad de un algoritmo transversal diferente.
- Observer: Los objetos interesados en un evento determinado pueden cambiar.
- State: El comportamiento en un determinado estado o la máquina de estados (estados y transiciones) de un determinado módulo cambia.
- Strategy: un algoritmo cambia.
- Template: los pasos concretos en un algoritmo cambian.
- Visitor: Se deben agregar nuevas operaciones a una estructura de clases de herencia más o menos estática dada.

DRY

Este principio (conocido como DRY) se explica por sí mismo: debes evitar al máximo grado posible la repetición de código. Partiendo de este principio se han creado un montón de maneras de reutilizar lo que ya hemos programado, piénsalo un poco: funciones, módulos, bibliotecas, clases, prototipos, herencia, composición, macros, saltos (goto).

Estas son sólo algunas maneras de evitar la repetición, claro, cada una de las estrategias anteriores lo logra a su manera y añade otras ventajas y desventajas (Importance of software design, 2021).

Importancia de no repetir código

- **Hace el código más mantenible.** Evitar la repetición de código permite que si alguna vez cambia la funcionalidad que estás repitiendo, no lo tengas que hacer en todos los lugares en los que lo repetiste.
- **Reduce el tamaño del código.** Esto lo hace más legible y entendible, porque hay menos código que entender. Los procesos para evitar la repetición implican nombrar el pedazo de código que estás reutilizando para identificarlo, esto hace el código más legible si lo nombraste bien.
- **Ahorra tiempo.** Al tener pedazos de código disponibles para reutilizarlos, en el futuro estás más preparado para lograr lo mismo en menos tiempo.

KISS

Este principio establece que:

- Nuestro software en general debería tener tan pocos componentes (y por lo tanto líneas) como sea posible.
- No deberíamos tener funcionalidades que no se ocupen actualmente “por si en el futuro se ocupan”.
- La documentación debe tener la información estrictamente necesaria.
- El código debe ser lo más obvio y sencillo posible. Se deben evitar esas líneas que sólo sirven para presumir lo inteligente que eres.
- El diseño debe mantener la estructura simple, siempre que se pueda (Importance of software design, 2021).

Importancia de la simplicidad del código

Las siguientes son algunas de las razones que justifican la existencia de este principio:

- **Proyectos más mantenibles.** Hay mucho menos que explicar al mantener las cosas simples. El código es más fácil de mantener y actualizar.
- **Menos documentación.** Hay menos cosas raras que documentar al hacer el código fácil, sin trucos que nadie entiende.
- **Debugging más rápido.** Al reducir la complejidad se pueden encontrar los errores más rápidamente.
- **Mayor rendimiento económico.** Los tres efectos anteriores permiten que la inversión económica inicial en el código creado tenga mayores rendimientos.

Este es uno de los principios más difíciles de aplicar, si no es que el más difícil. Hacer algo simple para los demás requiere de pensar las cosas el tiempo suficiente, de tener la experiencia técnica necesaria para evitar intentar matar una mosca con un cañón (o investigar y tener la capacidad de aprender). A final de cuentas la simplicidad es la sofisticación más avanzada (Herramientas UML, 2021).

SOLID

Es un acrónimo que engloba el nombre de 5 principios, los cuales son:

Single Responsibility (Responsabilidad Única). Una entidad de software debería tener una sola responsabilidad, esto también se puede interpretar como “tener una y sólo una razón para cambiar”. En pocas palabras, tu componente/función/clase debería hacer muy bien una sola cosa.

Open/Closed (Abierto/Cerrado). Una entidad de software (este principio está dirigido a las clases), debería estar abierto a extensión (crecer sus funcionalidades con otras entidades externas) pero cerrado a modificación.

Liskov Substitution (Sustitución de Liskov). El principio de sustitución de Liskov habla de interfaces: si una entidad de software usa una clase, este debe ser capaz de usar clases derivadas de esta. Esto es muy parecido a la programación por contrato, en el que se establecen las interfaces antes de la implementación.

Interface Segregation (Segregación de interfaces). Los clientes (las entidades de software) que usan una entidad de software (una clase, originalmente), no deberían estar obligados a depender de métodos que no usan. Para resolver esto, interfaces de gran tamaño se deben segregar, es decir, romper, en otras más pequeñas.

Dependency Inversion. El principio de Inversión de Dependencias establece que los módulos de alto nivel, es decir, los más cercanos a las ideas humanas de lo que debe hacer el software, no deben depender de los de bajo nivel (los más cercanos a la implementación de los detalles para la computadora). Ambos deberían depender de las abstracciones del problema (interfaces). Además, los detalles de implementación deben depender de las abstracciones también. Se llama así porque en general la gente lo piensa al revés (Herramientas UML, 2021).



Curiosidades. - Robert Martin introduce SOLID como un acrónimo de varios principios ya conocidos a comienzos de los 2000 y veinte años más tarde sigue siendo es una lectura indispensable para cualquier desarrollador de software.

1.3 Importancia del diseño de software

"Sin un buen diseño de software, la programación es un arte de agregar errores a un archivo de texto vacío" - Louise Srygley

El buen diseño no se trata solo de código. Se trata de poder expresar ideas para su software con otros desarrolladores, otros equipos y sus clientes. Tener un diseño bien pensado hace que su software sea más fácil de implementar, reduce la necesidad de realizar cambios importantes más adelante y ahorra dolores de cabeza en el futuro.

El software mejor diseñado es más flexible. Por lo tanto, puede agregar un nuevo componente al software existente sin afectar el software existente.

El software bien diseñado aumenta la reutilización. Porque si sigue estrictamente los patrones de diseño, su software sería más modular, es decir. Que consta de pequeños componentes que realizan una sola tarea. Por lo tanto, estos pequeños componentes se pueden reutilizar fácilmente.

Fácil de entender. Siempre es una tarea complicada explicar los proyectos a los nuevos empleados / miembros del equipo. Pero si tiene un buen diseño y documentación, puede comunicar fácilmente la idea del software a su nuevo miembro del equipo.

Se aumenta la rentabilidad. El diseño puede afectar el costo del software. Para entender esto, considere una situación en la que usted y su equipo comenzaron a construir software basándose en algunas suposiciones, pero después de desarrollar el 50% del software, se da cuenta de que ha llegado a un callejón sin salida y no puede seguir adelante supuestos. Ahora, tendrá que volver a iniciar el software, lo que definitivamente sería muy costoso. Por lo tanto, si se ha centrado primero en el diseño, podría haber descubierto el callejón sin salida antes y haber ahorrado mucho tiempo, mano de obra y dinero. Recordemos siempre que "diseñar es mucho más rentable que desarrollar" (Diseño Estructurado de Sistemas, 2021).



Recuerde que. - Sin un buen diseño inicial cualquier desarrollo de software puede llegar a convertirse en una inversión poco o nada rentable en vista que debe ser rediseñado después o incluso reemplazado.

1.4 Herramientas para la construcción de proyectos de software

Existen diferentes herramientas en el mercado que permiten construir software de maneras mucho más prácticas, herramientas orientadas a la construcción, modelado y la gestión del proyecto en sí. Desde herramientas libres y gratuitas patrocinadas por grandes corporaciones, hasta empresas con productos dedicados a llevar todo el proceso de desarrollo desde la concepción a la implementación (Diseño Estructurado de Sistemas, 2021).

Herramientas para desarrollo de software

IDE

Un entorno de desarrollo integrado (IDE) es un sistema de software para el diseño de aplicaciones que combina herramientas del desarrollador comunes en una sola interfaz gráfica de usuario (GUI). Generalmente, un IDE cuenta con las siguientes características:

- **Editor de código fuente:** editor de texto que ayuda a escribir el código de software con funciones como el resaltado de la sintaxis con indicaciones visuales, el relleno automático específico del lenguaje y la comprobación de errores a medida que se escribe el código.
- **Automatización de compilación local:** herramientas que automatizan tareas sencillas e iterativas como parte de la creación de una compilación local del software para su uso por parte del desarrollador, como la compilación del código fuente de la computadora en un código binario, el empaquetado del código binario y la ejecución de pruebas automatizadas.
- **Depurador:** programa que sirve para probar otros programas y mostrar la ubicación de un error en el código original de forma gráfica (Diseño Estructurado de Sistemas, 2021).

¿Por es necesario utilizar un IDE?

Los IDE permiten que los desarrolladores comiencen a programar aplicaciones nuevas con rapidez, ya que no necesitan establecer ni integrar manualmente varias herramientas como parte del proceso de configuración. Tampoco es necesario que pasen horas aprendiendo a utilizar diferentes herramientas por separado, gracias a que todas están representadas en la misma área de trabajo. Esto resulta muy útil al incorporar desarrolladores nuevos, porque pueden confiar en un IDE para ponerse al día con los flujos de trabajo y las herramientas estándares de un equipo. De hecho, la mayoría de las características de los IDE están diseñadas para ahorrar tiempo, como el relleno inteligente y la generación automatizada del código, lo cual elimina la necesidad de escribir secuencias enteras de caracteres.

Otras funciones comunes del IDE se encargan de ayudar a los desarrolladores a organizar su flujo de trabajo y solucionar problemas. Los IDE analizan el código mientras se escribe, así que las fallas causadas por errores humanos se identifican en tiempo real. Gracias a que hay una sola GUI que representa todas las herramientas, los desarrolladores pueden ejecutar tareas sin tener que pasar de una aplicación a otra. El resaltado de sintaxis también es común en la mayoría de los IDE, y utiliza indicaciones visuales para distinguir la gramática en el editor de texto. Además, algunos IDE incluyen examinadores de objetos y clases, así como diagramas de jerarquía de clases para ciertos lenguajes (Object Oriented Design, 2021).

Tipos de IDE conocidos

Hay muchos casos de uso comerciales y técnicos distintos para los IDE, lo cual también significa que hay muchas opciones de IDE propietarios y open source en el mercado. En general, las características distintivas más importantes entre los IDE son las siguientes:

Cantidad de lenguajes compatibles: algunos IDE son compatibles con un solo lenguaje, así que son mejores para un modelo de programación específico. Por ejemplo, IntelliJ es conocido principalmente como un IDE de Java. Otros IDE admiten una gran variedad de lenguajes de manera conjunta, como el IDE Eclipse, que admite Java, XML, Python, entre otros (Object Oriented Design, 2021).

- **Sistemas operativos compatibles:** el sistema operativo de un desarrollador determinará qué tipos de IDE son viables (salvo que el IDE esté basado en la nube),

y estarán aún más limitados si la aplicación que se desarrolla está diseñada para el usuario final con un sistema operativo específico (como Android o iOS).

- **Características de automatización:** si bien la mayoría de los IDE incluyen tres funciones fundamentales (el editor de texto, la automatización de compilación y el depurador), muchos admiten funciones adicionales, como la refactorización, la búsqueda de código y las herramientas de integración e implementación continuas (CI/CD).
- **Impacto en el rendimiento del sistema:** es importante considerar el footprint del IDE en la memoria si el desarrollador desea ejecutar otras aplicaciones con uso intensivo de la memoria al mismo tiempo.
- **Complementos y extensiones:** algunos IDE incluyen una función para personalizar los flujos de trabajo de forma que se adapten a las necesidades y preferencias del desarrollador.



Recuerde que. - Actualmente existen editores de texto con muchas características que los acercan a las funcionalidades de un IDE, la mayoría haciendo uso de plugins o extensiones.

CI / CD

“Quality at Speed” es la nueva norma en el desarrollo de software.

Las empresas están avanzando hacia las metodologías DevOps y la cultura ágil para acelerar la velocidad de entrega y garantizar la calidad del producto. En DevOps, un ciclo de entrega continuo y automatizado es la columna vertebral que hace posible una entrega rápida y confiable.

Esto da como resultado la necesidad de herramientas adecuadas de integración continua y entrega continua (CI / CD). Una "buena" herramienta de CI / CD puede aprovechar el flujo de trabajo actual de los equipos para aprovechar mejor la función de automatización y crear una línea sólida de CI / CD, y dar a los equipos el impulso que necesitan para prosperar.

En la actualidad, Internet ofrece una gran variedad de herramientas para la integración continua. Todas tienen como objetivo ayudar al desarrollador en la implementación de

esta metodología, y lo hacen de diferentes modos y con la ayuda de características distintas. Pero estas herramientas no solo se diferencian unas de otras en cuanto a sus características, sino que también existe una gran variedad en lo que respecta a precios y licencias. Mientras que muchas de ellas son de código abierto y se encuentran disponibles de forma gratuita, otros fabricantes ofrecen herramientas comerciales. A continuación, te ofrecemos un resumen de las más utilizadas y examinamos sus características y funciones (Data Oriented Design, 2021).

Herramientas de CI/CD más utilizadas

Figura 4. Herramientas CI-CD

	 Jenkins	 circleci	 TeamCity	 Bamboo	 GitLab
Open source	Yes	No	No	No	No
Ease of use & setup	Medium	Medium	Medium	Medium	Medium
Built-in features	3/5	4/5	4/5	4/5	4/5
Integration	★★★★★	★★★★	★★★★★	★★★★	★★★★★
Hosting	On premise & Cloud	On premise & Cloud	On premise	On premise & Bitbucket as Cloud	On premise & Cloud
Free version	Yes	Yes	Yes	Yes	Yes
Build Agent License Pricing	Free	From \$39 per month	From \$299 one-off payment	From \$10 one-off payment	From \$4 per month per user
Supported OSs	Windows, Linux, macOS, Unix-like OS	Linux or MacOS	Windows, Linux, macOS, Solaris, FreeBSD and more	Windows, Linux, macOS, Solaris	Linux distributions: Ubuntu, Debian, CentOS, Oracle Linux



Videos. - Resumen de lo que implica integración continua y despliegue continuo (CI / CD) <https://www.youtube.com/watch?v=scEDHsr3APg>.

Herramientas para modelado del software.

El UML, o Lenguaje de Modelado Unificado, abstrae y visualiza sistemas de la programación orientada a objetos. El lenguaje de modelado es, por lo tanto, una herramienta práctica para los desarrolladores de programas y sistemas. Por un lado, permite crear diagramas claros para proyectos de software, y por otro, presentar sistemas de programación complejos de forma comprensible para las personas que no están familiarizadas con la temática. Por ejemplo, si deseas presentar un proyecto de software

al responsable de marketing, por medio de un diagrama UML puedes ofrecer una visión general de las características más importantes de la aplicación.

Encontrar la herramienta adecuada no es tan fácil, ya que, aunque existen innumerables proveedores de programas UML en la red, no todos ofrecen las mismas funciones. Algunas herramientas requieren una capacidad de memoria menor y ofrecen pocas funciones; otras pueden mapear cualquier tipo de diagrama y exportarlo a diferentes lenguajes de programación o importar un modelo desde un código existente. En contrapartida, muchos de estos programas no ofrecen ninguna función para intercambiar información sobre el proyecto dentro de un equipo.

El consorcio Object Management Group (abreviado como OMG), que especifica los estándares UML, recomienda que primero consideres exactamente lo que deseas representar con los diagramas UML. ¿Debe mostrarse la estructura o el comportamiento del sistema? Dependiendo de la respuesta, tendrás que elegir la herramienta de modelador UML más adecuada para tu proyecto (Herramientas UML, 2021).

Caso de empleo A: realizar bocetos o prototipos simples

En estos casos las herramientas UML gratuitas son una buena opción, aunque estas variantes tienen la desventaja de que las formas gráficas permanecen siempre como formas, sin poder desarrollarse o ampliarse. Sin embargo, incluso una herramienta UML gratuita atribuye un significado a estas formas: por ejemplo, un simple rectángulo con la etiqueta <<classifier>> simboliza la instancia de una clase (Herramientas UML, 2021).

Caso de empleo B: abstraer sistemas complejos

La tarea principal del UML es simplificar los sistemas complejos, haciéndolos comprensibles incluso para aquellos que no poseen conocimientos específicos en programación y código. Por lo tanto, una buena herramienta de diagramas UML también facilita el trabajo en equipo y la cooperación entre los diferentes departamentos. Dado que estas extensas herramientas rara vez son gratuitas, debes asegurarte de que son compatibles con la mayoría de versiones actuales de la especificación UML 2, ya que UML 2 ofrece más tipos de diagramas que UML 1 y te permite crear perfiles que se adaptan con mayor precisión a tus necesidades (Herramientas UML, 2021).

Caso de empleo C: crear código a partir de un modelo

Con algunas herramientas UML, los diagramas se pueden convertir a un lenguaje de programación determinado: a menudo trabajan con esquemas Java, C++, C# o XML. Algunas herramientas también reconstruyen diagramas UML a partir de código existente. Sin embargo, paradójicamente, algunas de estas herramientas no proporcionan la función de ingeniería inversa para los mismos lenguajes de programación que utilizan para crear el código a partir de diagramas UML (Herramientas UML, 2021).

Las herramientas de diagramas UML con ingeniería inversa generan diagramas a partir del código fuente y vuelven a convertir la versión revisada en código.

Caso de empleo D: desarrollar modelos para integrar nuevos procesos

Si deseas integrar software nuevo en tu sistema, podrías utilizar UML 2.0 para mostrar los espacios vacíos. Las herramientas UML facilitan a los arquitectos de software y programadores la búsqueda de puntos de conexión. Si deseas escribir tu propio programa, por ejemplo, puedes visualizar interfaces de diferentes componentes del sistema utilizando un diagrama de componentes. Los diagramas de clase y de objeto, así como aquellos de actividad, proporcionan una visión detallada de los elementos individuales y de sus beneficios cuando se trata de añadir aplicaciones al sistema que se va a integrar (Herramientas UML, 2021).

Herramientas para gestión de proyectos de software

Reunir a tu personal y prepararlo para trabajar en equipo como una unidad es esencial para el éxito de tus proyectos. Pero sin las herramientas adecuadas para la administración de esos proyectos, puedes estar duplicando esfuerzos en lugar de optimizándolos. Si tienes que dedicar tiempo a mantener una decena de hojas de cálculo e intentar ponerte al día con una bandeja de entrada desbordada, perderás la oportunidad de ser estratégico con tu tiempo. Ahí reside la importancia de utilizar las herramientas de gestión de proyectos adecuadas para el trabajo (Diseño a nivel de componentes, 2021).

Las herramientas de gestión de proyectos pueden variar de un equipo de trabajo a otro, pero se usan habitualmente como programas informáticos que permiten a los gestores de proyectos planificar, ejecutar y gestionar sus proyectos en una ubicación virtual centralizada (Diseño a nivel de componentes, 2021).

El software de gestión de proyectos se puede utilizar para lo siguiente:

- Planificación del proyecto
- Programación del proyecto
- Asignación de recursos y planificación de capacidad
- Presupuesto y seguimiento de los costes del proyecto
- Gestión de la calidad
- Almacenamiento e intercambio de documentación y registros del proyecto
- Elaboración y publicación de informes del proyecto
- Seguimiento del tiempo real dedicado a tareas del proyecto en comparación con la planificación
- Análisis de tendencias y pronósticos.

1.5 Herramientas para el control de versiones de software

Un controlador de versiones es un sistema que nos permite guardar un registro de las modificaciones que realizamos sobre un fichero o conjunto de ficheros a lo largo del tiempo de tal manera que sea posible recuperar versiones específicas más adelante (Diseño a nivel de componentes, 2021).

Importancia de usar un VCS

Un software de control de versiones (VCS) es una valiosa herramienta con numerosos beneficios para un flujo de trabajo de equipos de software de colaboración. Cualquier proyecto de software que tiene más de un desarrollador manteniendo archivos de código fuente debe usar un VCS. Además, los proyectos mantenidos por una sola persona se beneficiarán enormemente de su uso. Se puede decir que no hay una razón válida para privarse del uso de un VCS en cualquier proyecto moderno de desarrollo de software (Diseño a nivel de componentes, 2021).

Características fundamentales

Resolución de conflictos

Es muy probable que los miembros del equipo tengan la necesidad de realizar cambios en el mismo archivo de código fuente al mismo tiempo. Un VCS monitoriza y ayuda a poder resolver los conflictos entre varios desarrolladores.

Revertir y deshacer los cambios en el código fuente

Al empezar a monitorizar un sistema de archivos de códigos fuente, existe la posibilidad de revertir y deshacer rápidamente a una versión estable conocida.

Copia de seguridad externa del código fuente

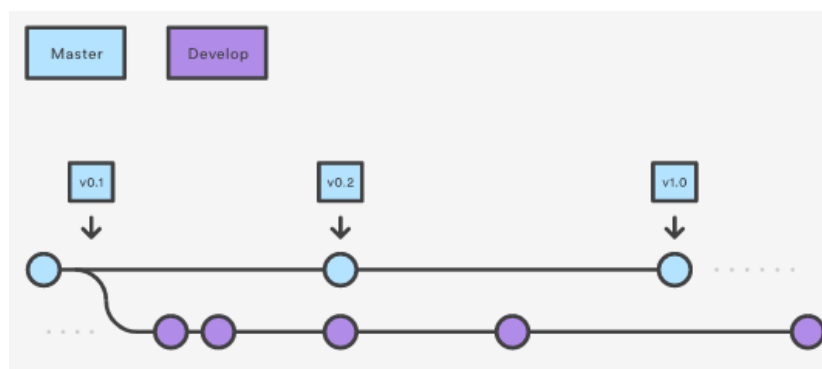
Se debe crear una instancia remota del VCS que se puede alojar de forma externa con un tercero de confianza y con ello, se conservará una copia del código fuente.

Gitflow

El flujo de trabajo de Gitflow es un diseño de flujo de trabajo de Git que fue publicado por primera vez y popularizado por Vincent Driessen en nvie. El flujo de trabajo de Gitflow define un modelo estricto de ramificación diseñado alrededor de la publicación del proyecto. Proporciona un marco sólido para gestionar proyectos más grandes.

Ramas de desarrollo y maestras

Figura 5. Esquema Gitflow



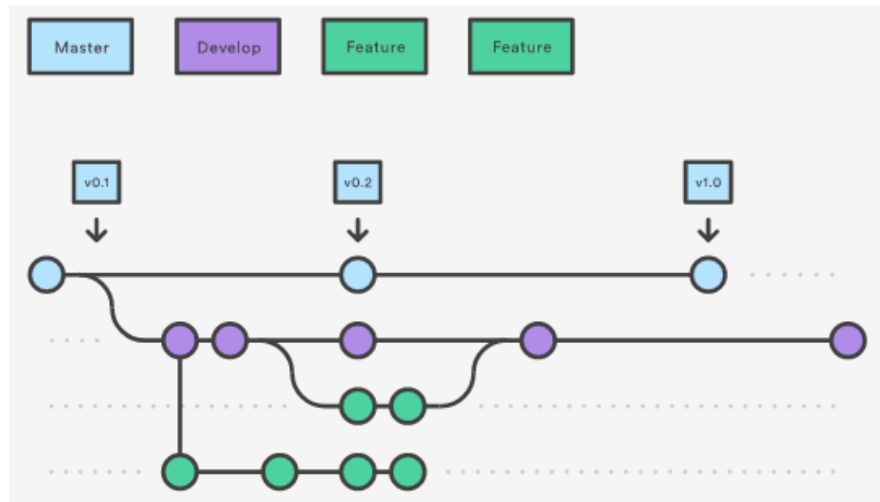
En vez de una única rama maestra, este flujo de trabajo utiliza dos ramas para registrar el historial del proyecto. La rama maestra almacena el historial de publicación oficial y la rama de desarrollo sirve como rama de integración para funciones. Asimismo, conviene etiquetar todas las confirmaciones de la rama maestra con un número de versión (Diseño a nivel de componentes, 2021).

Figura 6. Código de publicación Gitflow

```
git branch develop
git push -u origin develop
```

Ramas de función

Figura 7. Esquema de ramas de función gitflow



Cada nueva función debe estar en su propia rama, que se puede enviar al repositorio central para copia de seguridad/colaboración. Sin embargo, en vez de ramificarse de la maestra, las ramas de función utilizan la de desarrollo como rama primaria. Cuando una función está completa, se vuelve a fusionar en la de desarrollo. Las funciones nunca deben interactuar directamente con la maestra (Gitflow workflow, 2021).

Figura 8. Código de chequeo - gitflow

```
git checkout develop
git checkout -b feature_branch
```

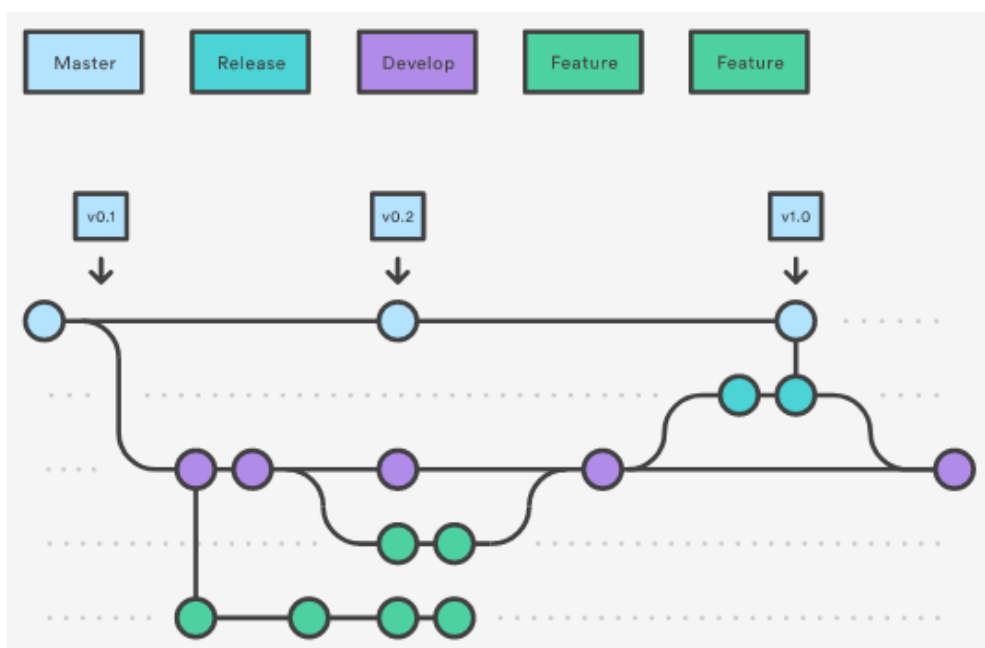
Cuando hayas terminado con el trabajo de desarrollo en la función, el siguiente paso es fusionar feature_branch en develop.

Figura 9. Código de fusión - gitflow

```
git checkout develop
git merge feature_branch
```

Ramas de publicación

Figura 10. Esquema de publicación gitflow



Una vez que el desarrollo ha adquirido suficientes funciones para una publicación (o se está acercando una fecha de publicación predeterminada), bifurcas una rama de versión a partir de una de desarrollo. Al crear esta rama, se inicia el siguiente ciclo de publicación, por lo que no pueden añadirse nuevas funciones tras este punto; solo las soluciones de errores, la generación de documentación y otras tareas orientadas a la publicación deben ir en esta rama. Una vez que esté lista para el lanzamiento, la rama de publicación se fusiona en la maestra y se etiqueta con un número de versión. Además, debería volver a fusionarse en la de desarrollo, que podría haber progresado desde que se inició la publicación (Gitflow workflow, 2021).

Figura 11. Código de lanzamiento - gitflow

```
git checkout develop
git checkout -b release/0.1.0
```

Una vez que la publicación esté lista para su lanzamiento, se fusionará en la maestra y la de desarrollo; a continuación, se eliminará la rama de publicación. Es importante volver a fusionarla en la de desarrollo porque podrían haberse añadido actualizaciones críticas a la rama de publicación y las nuevas funciones tienen que poder acceder a ellas. Si tu

organización enfatiza la revisión de código, sería el lugar ideal para una solicitud de incorporación de cambios (Gitflow workflow, 2021).

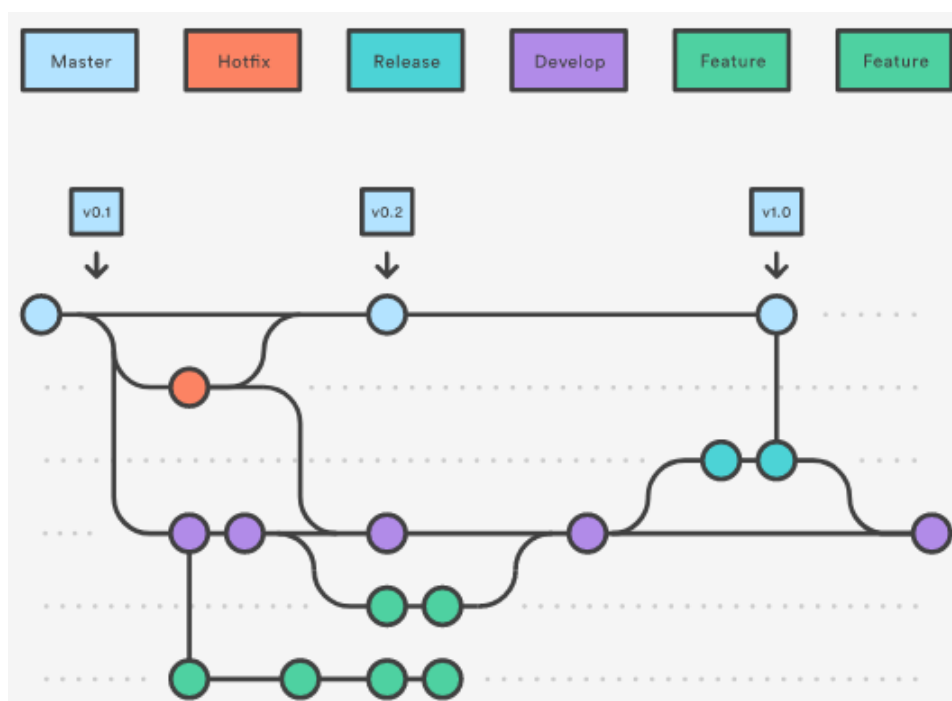
Para finalizar una rama de publicación, utiliza los siguientes métodos:

Figura 12. Código de finalización - gitflow

```
git checkout master
git merge release/0.1.0
```

Ramas de corrección

Figura 12. Esquema de corrección - gitflow



Las ramas de mantenimiento o "corrección" (hotfix) se utilizan para reparar rápidamente las publicaciones de producción. Las ramas de corrección son muy similares a las ramas de publicación y a las de función, salvo porque se basan en la maestra en vez de la de desarrollo. Es la única rama que debería bifurcarse directamente a partir de la maestra. Una vez que la solución esté completa, debería fusionarse en la maestra y la de desarrollo (o la rama de publicación actual), y la maestra debería etiquetarse con un número de versión actualizado (Gitflow workflow, 2021).

Tener una línea de desarrollo específica para la solución de errores permite que tu equipo aborde las incidencias sin interrumpir el resto del flujo de trabajo ni esperar al siguiente

ciclo de publicación. Puedes considerar las ramas de mantenimiento como ramas de publicación ad hoc que trabajan directamente con la maestra.

Figura 13. Código de corrección - gitflow

```
git checkout master  
git checkout -b hotfix_branch
```

Al igual que al finalizar una rama de publicación, una rama de corrección se fusiona tanto en la maestra como en la de desarrollo.

Figura 14. Código de fusión 2 - gitflow

```
git checkout master  
git merge hotfix_branch  
git checkout develop  
git merge hotfix_branch  
git branch -D hotfix_branch
```



Sabías que. - Git fue diseñado por Linus Torvalds, creador de Linux, la primera versión de Git fue liberada en el año 2005 buscando una forma de registrar los cambios en archivos y coordinar la colaboración sobre los mismos.



Tema 2: Paradigmas de diseño

2.1 Diseño estructurado

El diseño estructurado de sistemas se ocupa de la identificación, selección y organización de los módulos y sus relaciones. Se comienza con la especificación resultante del proceso de análisis, se realiza una descomposición del sistema en módulos estructurados en jerarquías, con características tales que permitan la implementación de un sistema que no requiera elevados costos de mantenimiento.

La idea original del diseño estructurado fue presentada en la década de los '70, por Larry Constantine, y continuada posteriormente por otros autores: Myers, Yourdon y Stevens (Diseño a nivel de componentes, 2021).

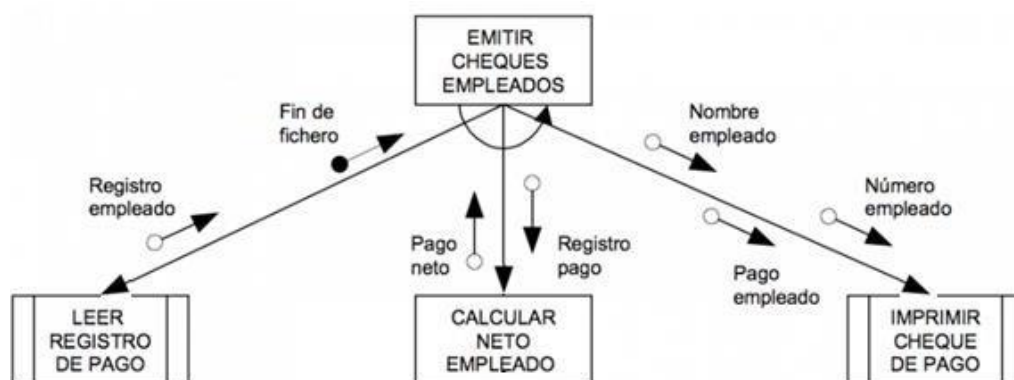
Introducción

El diseño estructurado es un enfoque disciplinado de la transformación de qué es necesario para el desarrollo de un sistema, a cómo deberá ser hecha la implementación.

La definición anterior implica que: el análisis de requerimientos del usuario (determinación del qué) debe preceder al diseño y que, al finalizar el diseño se tendrá medios para la implementación de las necesidades del usuario (el cómo), pero no se tendrá implementada la solución al problema (Diseño Estructurado de Sistemas, 2021). Cinco aspectos básicos pueden ser reconocidos:

1. Permitir que la forma del problema guíe a la forma de la solución. Un concepto básico del diseño de arquitecturas es: las formas siempre siguen funciones.
2. Intentar resolver la complejidad de los grandes sistemas a través de la segmentación de un sistema en cajas negras, y su organización en una jerarquía conveniente para la implementación.
3. Utilizar herramientas, especialmente gráficas, para realizar diseños de fácil comprensión. Un diseño estructurado usa diagramas de estructura (DE) en el diseño de la arquitectura de módulos del sistema y adiciona especificaciones de los módulos y cuplas (entradas y salidas de los módulos), en un Diccionario de Datos (DD).
4. Ofrecer un conjunto de estrategias para derivar el diseño de la solución, basándose en los resultados del proceso de análisis.
5. Ofrecer un conjunto de criterios para evaluar la calidad de un diseño con respecto al problema a ser resuelto, y las posibles alternativas de solución, en la búsqueda de la mejor de ellas.

Figura 15. Esquema paradigma de diseño



Un diagrama de estructura permite modelar un programa como una jerarquía de módulos. Cada nivel de la jerarquía representa una descomposición más detallada del módulo del nivel superior. La notación usada se compone básicamente de tres símbolos que son Módulos, Invocaciones y Cuplas.

Módulos

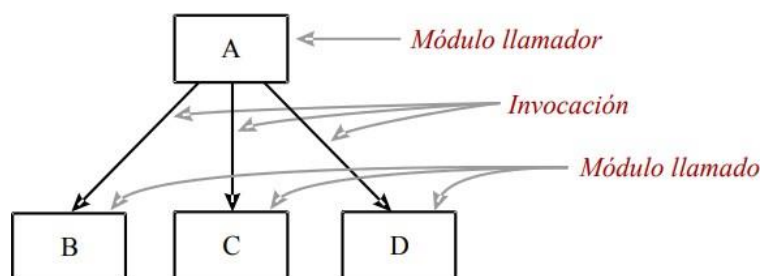
Un módulo es un conjunto de instrucciones que ejecutan alguna actividad, un procedimiento o función. Desde un punto de vista práctico, un módulo es una colección de instrucciones de un programa con cuatro características básicas:

1. **Entradas y Salidas:** lo que un módulo recibe en una invocación y lo que retorna como resultado.
2. **Función:** las actividades que un módulo hace con la entrada para producir la salida.
3. **Lógica Interna:** por la cual se ejecuta la función.
4. **Estado Interno:** su área de datos privada, datos para los cuales sólo el módulo hace referencia.

Invocaciones

En la realidad, los módulos no son realmente cajas negras. Los diagramas de estructura muestran las invocaciones que un módulo hace a otros módulos. Estas invocaciones son diseñadas como una flecha que sale del módulo llamador y apunta al módulo llamado. Una caja negra no permite que se observe su interior y, las invocaciones que un módulo hace son componentes de su estructura interna. De todas formas, se dice que un módulo es una caja casi negra o caja gris porque ella permite que se observe solo las invocaciones (Importance of software design, 2021).

Figura 16. Esquema de invocaciones

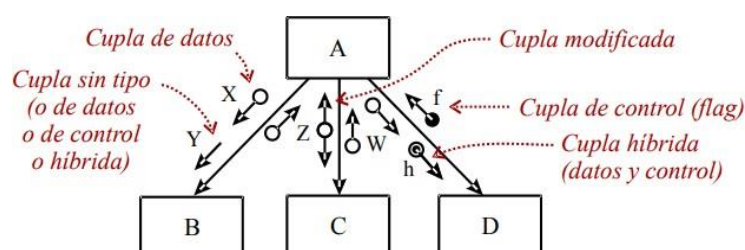


Los diagramas de estructura no tienen especificado el orden de invocación de los módulos invocados, a pesar de esto se recomienda que siempre que fuese posible, se siga un orden de izquierda a derecha (si esto no produce que se crucen flechas) que se corresponde con el orden de invocación, y permitiendo un orden de lectura que es patrón en la mayoría de los idiomas.

Comunicación entre Módulos (Cuplas)

Cuando una función o un procedimiento, en un lenguaje convencional, es invocado, comúnmente un conjunto de argumentos es comunicado y, en el caso de las funciones, también se espera que retorne un resultado. Estos datos comunicados en una invocación son modelados por medio de flechas, sobre el símbolo de invocación, llamadas cuplas (Importance of software design, 2021).

Figura 17. Esquema de Cuplas



Cupla de Datos



Una cupla de datos transporta datos “puros” a un módulo. No es necesario conocer la lógica interna del módulo receptor, para determinar los valores válidos de la cupla (ej.: número de cuenta, saldo, tabla de movimientos) (Software Design & Architecture map, 2021).

Cupla Modificada



Con una flecha doble (apuntando al módulo llamador y al módulo llamado) se especifica un argumento enviado a un módulo que deberá modificar su valor, fijando un nuevo valor disponible para el módulo llamador (en la implementación, se precisará que el lenguaje posea un mecanismo de pasaje de parámetros por referencia) (ej.: el buffer enviado a un módulo de lectura de un archivo) (Software Design & Architecture map, 2021).

Cupla de Resultados



Existen módulos que retornan valores sin la necesidad de que estén inicializados en el momento que se invocan. Estos casos son dos:

1. Cuplas similares al tipo Modificada cuyos valores previos a la invocación del módulo NO se utilizan para calcular el valor de retorno. Si bien en Pascal se implementa como las “Cuplas Modificadas”, es conveniente documentarlas en el DE como “de Resultados” por cuestiones de claridad.
2. Si el Módulo en cuestión es una función (retorna un valor), se debe documentar este valor de retorno como “Cupla de Resultado” cuyo nombre se corresponderá con el nombre de la función (Software Design & Architecture map, 2021).

2.2 Diseño orientado a objetos

Introducción

El diseño orientado a objetos (OOD) es un enfoque del diseño de software y se define como el proceso de planificación de un sistema de objetos en interacción con el propósito de resolver un problema de software.

El diseño orientado a objetos (OOD) sirve como parte del proceso de estilo de vida de la programación orientada a objetos (OOP). Es principalmente el proceso de utilizar una metodología de objetos para diseñar una aplicación o sistema informático. Esta técnica permite la implementación de un software basado en los conceptos de objetos. Además, es un concepto que obliga a los programadores a planificar su código para tener un programa que fluya mejor.

Se debaten los orígenes del diseño orientado a objetos (OOD), pero los primeros lenguajes que lo admitieron incluyeron Simula y SmallTalk. El término no se hizo popular hasta que Grady Booch escribió el primer artículo titulado Diseño orientado a objetos, en 1982. El objetivo principal de este tipo de diseño de software es definir las clases y sus relaciones, que son necesarias para construir un sistema que cumpla con los requisitos contenidos en las Especificaciones de Requisitos de Software (Diseño Estructurado de Sistemas, 2021).

Además, es la disciplina de definir los objetos y sus interacciones para resolver un problema que fue identificado y documentado durante el Análisis Orientado a Objetos

(OOA). En resumen, el diseño orientado a objetos (OOD) es un método de diseño que abarca el proceso de descomposición orientada a objetos y una notación para representar modelos lógicos y físicos del sistema en diseño. Las otras características del Diseño Orientado a Objetos son las siguientes:

- Los objetos son abstracciones del mundo real o entidades del sistema y se administran a sí mismos.
- Los objetos son independientes y se encuentran en un estado encapsulado e información de representación.
- La funcionalidad del sistema se expresa en términos de servicios de objetos.
- Se eliminan las áreas de datos compartidos.
- La comunicación entre objetos se realiza a través del paso de mensajes.
- Los objetos pueden estar distribuidos y pueden ejecutarse secuencialmente o en paralelo (Diseño Estructurado de Sistemas, 2021).

Proceso de diseño orientado a objetos

Comprender el proceso de cualquier tipo de actividad relacionada con el software simplifica su desarrollo para el desarrollador, programador y evaluador de software. Ya sea que esté ejecutando pruebas funcionales o haciendo un informe de prueba, todas y cada una de las acciones tienen un proceso que deben seguir los miembros del equipo. Del mismo modo, el diseño orientado a objetos (OOD) también tiene un proceso definido, que si no se sigue rigurosamente, puede afectar el rendimiento y la calidad del software. Por lo tanto, para ayudar al equipo de desarrolladores y programadores de software, aquí está el proceso de Diseño Orientado a Objetos (OOD):

Para diseñar clases y sus atributos, métodos, asociaciones, estructura e incluso protocolo, se aplica el axioma de diseño.

- El diagrama de clases de UML estático se redefine y completa agregando detalles.
- Los atributos se refinan.
- Los protocolos y métodos se diseñan utilizando un diagrama de actividad UML para representar el algoritmo de métodos.
- Si es necesario, redefine las asociaciones entre clases y refine la jerarquía de clases y el diseño con herencia.
- Itere y refine de nuevo (Diseño Estructurado de Sistemas, 2021).

Diseña la capa de acceso

- Cree clases espejo, es decir, para cada clase empresarial identificada y creada, cree una clase de acceso.

Identificar la relación de clases de la capa de acceso.

Simplifique las clases y sus relaciones. El objetivo principal aquí es eliminar clases y estructuras redundantes.

- Clases redundantes: los programadores deben recordar no colocar dos clases que realicen solicitudes de traducción similares y actividades de resultados de traducción. Simplemente deben seleccionar uno y eliminar el otro.
- Clases de métodos: revise las clases que constan de solo uno o dos métodos, para ver si pueden eliminarse o combinarse con las clases existentes.

Itere y refine de nuevo.

Diseñe las clases de capa de vista.

- Diseñe la interfaz de usuario de nivel macro, mientras identifica los objetos de la capa de vista.
- Diseñe la interfaz de usuario de nivel micro.
- Pruebe la usabilidad y la satisfacción del usuario.
- Iterar y perfeccionar.

Al final del proceso, repita todo el diseño. Vuelva a aplicar los axiomas de diseño y, si es necesario, repita los pasos anteriores nuevamente (Importance of software design, 2021).

Conceptos de diseño orientado a objetos:

En el diseño orientado a objetos (OOD), los conceptos independientes de la tecnología en el modelo de análisis se asignan a las clases de implementación, se identifican las restricciones y se diseñan las interfaces, lo que da como resultado un modelo para el dominio de la solución. En resumen, se construye una descripción detallada para especificar cómo se construirá el sistema sobre tecnologías concretas. Además, el Diseño Orientado a Objetos (OOD) sigue algunos conceptos para lograr estos objetivos, cada uno de los cuales tienen un rol específico y tiene mucha importancia. Estos conceptos se definen en detalle a continuación:

Encapsulación: se trata de un acoplamiento o asociación estrecho de la estructura de datos con los métodos o funciones que actúan sobre los datos. Esto se conoce básicamente como clase u objeto (el objeto suele ser la implementación de una clase).

Protección de datos: la capacidad de proteger algunos componentes del objeto de entidades externas. Esto se realiza mediante palabras reservadas del lenguaje para permitir que una variable se declare como privada o protegida para la clase propietaria.

Herencia: esta es la capacidad de una clase para extender o anular la funcionalidad de otra clase. Esta llamada clase secundaria tiene una sección completa que es la clase principal y luego tiene su propio conjunto de funciones y datos.

Interfaz: una definición de funciones o métodos, y su firma que están disponibles para su uso, así como para manipular una instancia determinada de un objeto.

Polimorfismo: esta es la capacidad de definir diferentes funciones o clases que tienen el mismo nombre, pero que toman diferentes tipos de datos (Importance of software design, 2021).

Ventajas del diseño orientado a objetos:

Los beneficios de este enfoque de diseño de software son numerosos. Por lo tanto, proporcionamos aquí algunas de las otras ventajas de usar el diseño orientado a objetos (OOD).

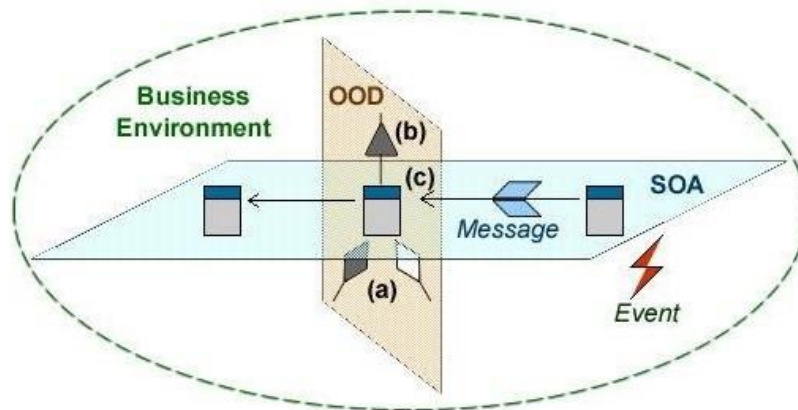
- Más fácil de mantener los objetos.
- Los objetos pueden entenderse como entidades independientes.
- Los objetos son componentes reutilizables apropiados.
- Para algunos sistemas, puede haber un mapeo obvio de entidades reales a objetos del sistema (Encapsulate the Concept that Varies, 2021).

2.3 Diseño orientado a eventos

Los principios orientados a objetos se pueden simplificar en torno a tres temas centrales:

(a) ocultación de información y acoplamiento entre estructuras y métodos; **(b)** herencia entre tipos; **(c)** comunicación a través de interfaces y polimorfismo.

Figura 18. Esquema de Diseño Orientado a Objetos

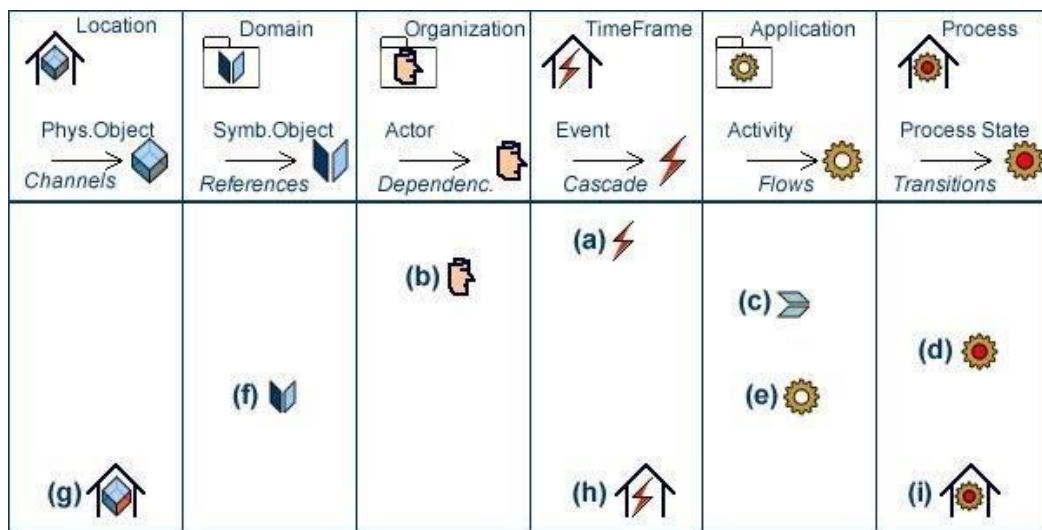


De estos, el encapsulado y la herencia son específicos del diseño de software, pero los mecanismos de comunicación también son el núcleo de las arquitecturas orientadas a servicios. Considerando los mensajes como contrapartes lógicas del sistema de los eventos comerciales, el análisis orientado a eventos debería ayudar a alinear los procesos comerciales con las capacidades de los sistemas (Encapsulate the Concept that Varies, 2021).

Desde la perspectiva de los procesos de negocio, los eventos señalan cambios en el estado de las actividades, los objetos o las expectativas. Dado que los sistemas de soporte están destinados a hacer frente a esos cambios, el análisis de los requisitos comerciales podría proceder de los eventos correspondientes:

- Los eventos comerciales se definen con respecto a los plazos **(a)** y las fuentes que deben autenticarse y autorizarse **(b)**.
- Los cambios desencadenantes deben describirse mediante mensajes con respecto a su alcance funcional **(c)** y operativo **(d)**.
- La lógica empresarial **(e)** y las entidades **(f)** a menudo se comparten entre aplicaciones y, por lo tanto, se definen mejor de forma independiente.
- Los cambios internos (mismo espacio y período de tiempo) están ocultos.
- Los cambios desencadenados (externos) se definen con respecto a los marcos de tiempo **(h)**, los procesos **(d)** y los dispositivos **(g)**.

Figura 19. Modelado OO



Estas facetas se pueden alinear con las de diseño OO, con **(c)** y **(d)** para comunicación, **(e)** y **(f)** para encapsulación. Desde una perspectiva más amplia, también encajan con el creciente enfoque en aplicaciones impulsadas por eventos y arquitecturas orientadas a servicios.

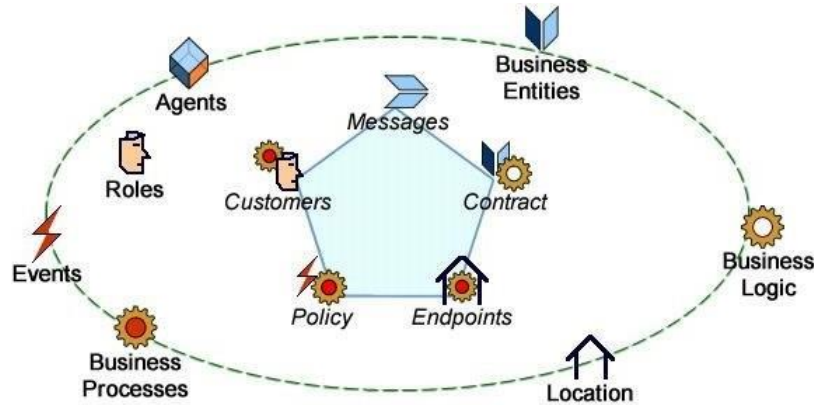
Al mover la lógica empresarial a un segundo plano, el análisis basado en eventos fomenta el polimorfismo a nivel empresarial con los beneficios correspondientes:

- Con respecto a los procesos de negocio, los eventos vienen con requisitos funcionales y operativos establecidos independientemente de la lógica de negocio que se llevará a cabo: desencadenante (qué ha cambiado), rol (quién está solicitando) y semántica de comunicación de mensajes (cuando se supone que el sistema debe lidiar con el evento).
- Con respecto a las capacidades del sistema, los mensajes se pueden usar para alinear los eventos comerciales (externos) con los del sistema (internos), independientemente de las entidades comerciales y la lógica (qué se debe hacer y cómo).
- Con respecto a la arquitectura y el diseño, ese enfoque es mantener los principios de OO tratando por separado las solicitudes polimórficas (interfaces) y la lógica empresarial (métodos) (SOLID Principles, 2021).

Esos beneficios aparecen claramente cuando las capacidades se realizan mediante servicios definidos con respecto a los procesos comerciales (clientes), los objetos

comerciales (mensajes), la lógica comercial (contrato) y las operaciones comerciales (política).

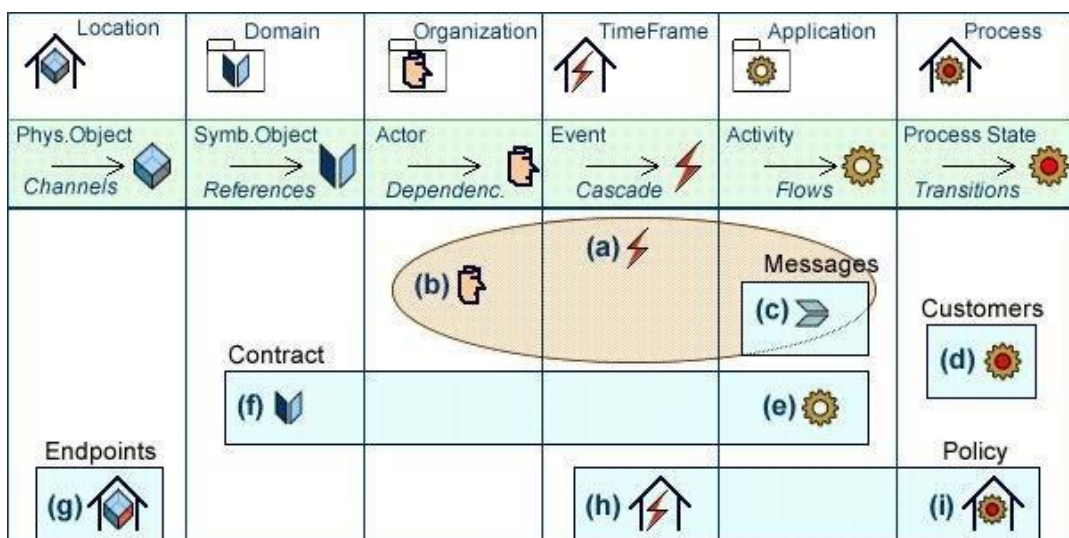
Figura 20. Modelo OO



Con eventos establecidos como anclajes de modelado, los casos de uso pueden proporcionar la relación de modelado entre procesos y capacidades funcionales:

- Eventos desencadenantes **(a)** mapee los cambios en los entornos empresariales (eventos externos) a los cambios en los objetos del sistema (eventos internos).
- Los actores **(b)** asignan roles en la organización a los usuarios del sistema.
- Los mensajes **(c)** asignan la semántica de los procesos comerciales a la semántica de las aplicaciones **(e)** y los dominios **(f)**.

Figura 21. Esquema de Eventos OO



Sobre esa base, el principal objetivo del análisis orientado a eventos sería distinguir entre la comunicación y la semántica empresarial, la primera se ocupa de las interacciones y la segunda de la lógica empresarial (SOLID Principles, 2021).

2.4 Diseño orientado al nivel de componentes

El diseño de datos a nivel de componentes se centra en la representación de estructuras de datos a las que se accede directamente a través de uno o más componentes del software.

Esta orientación transforma los elementos estructurales de la arquitectura del software en una descripción de sus componentes en cuanto a procedimiento. La información obtenida a partir de los modelos basados en clase, flujo y comportamiento sirve como la base para diseñar los componentes.

Un componente es un bloque de construcción de software de cómputo. Desde la visión orientada a objetos, un componente contiene un conjunto de clases que colaboran. Cada clase dentro de un componente se elabora por completo para que incluya todos los atributos y operaciones relevantes para su implantación (Software Design & Architecture map, 2021).

En el contexto de la ingeniería de software tradicional, un componente es un elemento funcional de un programa que incorpora la lógica del procesamiento, las estructuras de datos internas que se requieren para implantar la lógica del procesamiento y una interfaz que permite la invocación del componente y el paso de los datos. Dentro de la arquitectura del software se encuentra un componente tradicional, también llamado módulo, que tiene tres funciones importantes:

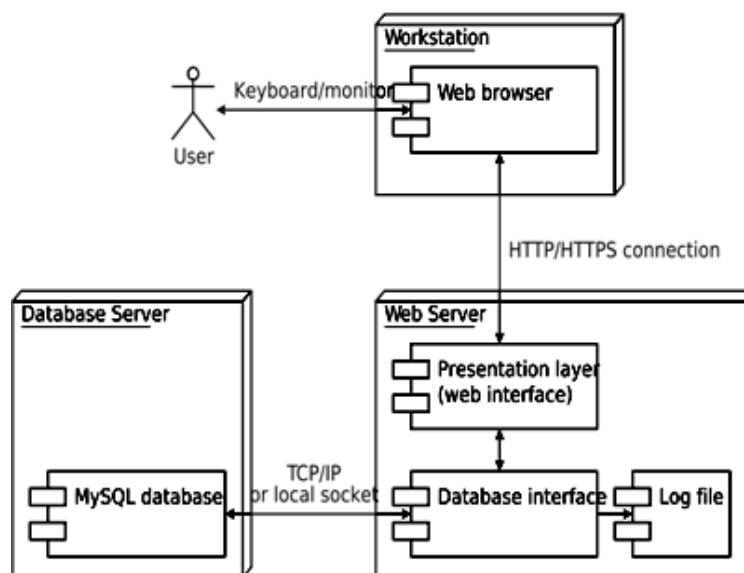
- 1) Como componente de control que coordina la invocación de todos los demás componentes del dominio del problema
- 2) Como componente del dominio del problema que implanta una función completa o parcial que requiere el cliente
- 3) Como componente de infraestructura que es responsable de las funciones que dan apoyo al procesamiento requerido en el dominio del problema.

Los componentes son entidades desplegables. Es decir, no son compilados en un programa de aplicación, sino que se instalan directamente sobre una plataforma de

ejecución. Los métodos y atributos definidos en sus interfaces pueden ser accedidos por otros componentes.

Aquí tenemos dos nodos, el cliente y el servidor, cada uno de ellos contiene componentes. El componente del cliente utiliza una interface de uno de los componentes del servidor. Se muestra la relación existente entre los nodos. A esta relación podríamos asociarle un estereotipo para indicar qué tipo de conexión disponemos entre el cliente y el servidor, así como modificar su cardinalidad para indicar que soportamos diversos clientes. Como los componentes pueden residir en más de un nodo, podemos situar el componente de forma independiente, sin que pertenezca a ningún nodo y relacionarlo con los nodos en los que se sitúa (Software Design & Architecture map, 2021).

Figura 22. Diseño de Patrones



2.5 Diseño centrado en los datos

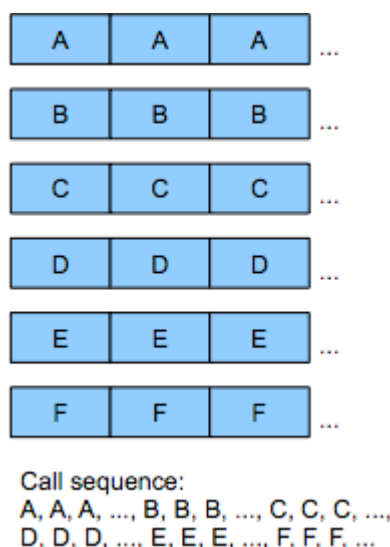
La programación orientada a datos se acerca a la codificación de una manera ligeramente diferente a la programación orientada a objetos. En lugar de objetos, todo son datos y se puede actuar, sobre todo. Esto separa la funcionalidad y los datos. Ya no están entrelazados por un conjunto de reglas específico. En DOP, sus funciones son de uso general y se aplican a grandes cantidades de datos. Idealmente, debería estructurar los datos lo más cerca posible de los datos de salida para garantizar que la función en sí realice el menor esfuerzo. En otras palabras, el diseño orientado a datos cambia la

perspectiva de la programación de los objetos a los datos en sí: el tipo de datos, cómo se colocan en la memoria y cómo se leerán y procesarán (Data Oriented Design, 2021).

Datos ideales

Para lograr el mejor diseño de datos posible, es útil dividir cada objeto en los diferentes componentes y agrupar los componentes del mismo tipo en la memoria, independientemente del objeto del que provengan. Esta organización da como resultado grandes bloques de datos homogéneos, que nos permiten procesar los datos de forma secuencial (Data Oriented Design, 2021).

Figura 23. Estructura de datos ideales



Una razón clave por la que el diseño orientado a datos es tan poderoso es porque funciona muy bien en grandes grupos de objetos. OOP, por definición, trabaja en un solo objeto.

Ventajas del diseño orientado a datos

Paralelización

En estos días, no hay forma de evitar el hecho de que tenemos que lidiar con múltiples núcleos. Cualquiera que haya intentado tomar algún código OOP y paralelizarlo puede dar fe de lo difícil, propenso a errores y posiblemente no muy eficiente que es. A menudo, termina agregando muchas primitivas de sincronización para evitar el acceso simultáneo a los datos de varios subprocesos y, por lo general, muchos de los subprocesos terminan inactivos durante bastante tiempo esperando que se completen otros subprocesos. Como

resultado, la mejora del rendimiento puede ser bastante decepcionante (Diseño a nivel de componentes, 2021).

Cuando aplicamos el diseño orientado a datos, la paralelización se vuelve mucho más simple: tenemos los datos de entrada, una pequeña función para procesarlos y algunos datos de salida. Podemos tomar fácilmente algo así y dividirlo entre varios subprocesos con una sincronización mínima entre ellos. Incluso podemos ir más allá y ejecutar ese código en procesadores con memoria local (como las SPU en el procesador Cell) sin tener que hacer nada diferente (Diseño a nivel de componentes, 2021).

Utilización de caché.

Además de utilizar múltiples núcleos, una de las claves para lograr un gran rendimiento en el hardware moderno, con sus canales de instrucción profundos y sistemas de memoria lentos con múltiples niveles de cachés, es tener acceso a la memoria amigable con el caché. El diseño orientado a datos da como resultado un uso muy eficiente de la caché de instrucciones porque el mismo código se ejecuta una y otra vez. Además, si colocamos los datos en grandes bloques contiguos, podemos procesar los datos secuencialmente, obteniendo un uso casi perfecto de la caché de datos y un gran rendimiento. Posibles optimizaciones. Cuando pensamos en objetos o funciones, tendemos a atascarnos en la optimización a nivel de función o incluso de algoritmo; Reordenar algunas llamadas a funciones, cambiar el método de ordenación o incluso reescribir código C con ensamblador (Importance of software design, 2021).

Ese tipo de optimización es ciertamente beneficioso, pero si pensamos primero en los datos, podemos dar un paso atrás y realizar optimizaciones más grandes e importantes. Recuerde que todo lo que hace un juego es transformar algunos datos (activos, entradas, estado) en otros datos (comandos gráficos, nuevos estados del juego). Teniendo en cuenta ese flujo de datos, podemos tomar decisiones más inteligentes y de mayor nivel en función de cómo se transforman los datos y cómo se utilizan. Ese tipo de optimización puede ser extremadamente difícil y llevar mucho tiempo de implementar con métodos de programación orientada a objetos más tradicionales (Importance of software design, 2021).

Modularidad

A menudo existe un conflicto entre las técnicas que mejoran el rendimiento y las técnicas que ayudan a la legibilidad y la facilidad de desarrollo. Por ejemplo, reescribir algún código en lenguaje ensamblador puede resultar en un aumento del rendimiento, pero generalmente hace que el código sea más difícil de leer y mantener (Importance of software design, 2021).

Afortunadamente, el diseño orientado a datos es beneficioso tanto para el rendimiento como para la facilidad de desarrollo. Cuando escribe código específicamente para transformar datos, termina con funciones pequeñas, con muy pocas dependencias en otras partes del código. El código base termina siendo muy "plano", con muchas funciones hoja sin muchas dependencias. Este nivel de modularidad y falta de dependencias facilita mucho la comprensión, la sustitución y la actualización del código (Importance of software design, 2021).

Pruebas

La última gran ventaja del diseño orientado a datos es la facilidad de prueba, escribir pruebas unitarias para verificar las interacciones de los objetos no es trivial. Necesita configurar simulacros y probar cosas indirectamente. Por otro lado, cuando se trata directamente con datos, no podría ser más fácil escribir pruebas unitarias: cree algunos datos de entrada, llame a la función de transformación y verifique que los datos de salida sean los que esperamos. No hay nada más que eso. En realidad, esto es una gran ventaja y hace que el código sea extremadamente fácil de probar, ya sea que esté realizando un desarrollo basado en pruebas o simplemente escribiendo pruebas unitarias después del código (Importance of software design, 2021).

Inconvenientes del diseño orientado a datos

El diseño orientado a datos no es la solución mágica para todos los problemas en el desarrollo de juegos. Ayuda enormemente a escribir código de alto rendimiento y hace que los programas sean más legibles y fáciles de mantener, pero tiene algunos inconvenientes.

El principal problema con el diseño orientado a datos es que es diferente a lo que la mayoría de los programadores están acostumbrados o aprenden en la escuela. Requiere girar nuestro modelo mental del programa noventa grados y cambiar nuestra forma de

pensar sobre él. Se necesita algo de práctica antes de que se convierta en una segunda naturaleza.

Además, debido a que es un enfoque diferente, puede ser un desafío interactuar con el código existente, escrito de una manera más OOP o procedimental. Es difícil escribir una sola función de forma aislada, pero siempre que pueda aplicar el diseño orientado a datos a todo un subsistema, debería poder obtener muchos de los beneficios (Encapsulate the Concept that Varies, 2021).



Videos. – El siguiente video contiene un breve resumen práctico acerca del diseño orientado a datos <https://www.youtube.com/watch?v=GY9RytdA1mA>.

Bibliografía

Software Design & Architecture map. (2021). Retrieved 29 April 2021, from <https://www.freecodecamp.org/news/software-design/>

SOLID Principles: The Software Developer's Framework to Robust & Maintainable Code. (2021). Retrieved 29 April 2021, from <https://khalilstemmler.com/articles/solid-principles/solid-typescript/>

Encapsulate the Concept that Varies. (2021). Retrieved 29 April 2021, from http://principles-wiki.net/principles:encapsulate_the_concept_that_varies

Importance of software design. (2021). Retrieved 02 Mayo 2021, from <https://medium.com/swlh/importance-of-software-design-7ffea48ede17>

Herramientas UML. (2021). Retrieved 02 Mayo 2021, from <https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/las-mejores-herramientas-uml/>

Diseño Estructurado de Sistemas. (2021). Retrieved 02 Mayo 2021, from <https://users.exa.unicen.edu.ar/catedras/prog1/sites/default/files/ApuntesDiagramaEstructura.pdf>

Object Oriented Design. (2021). Retrieved 02 Mayo 2021, from <https://www.professionalqa.com/object-oriented-design>

Data Oriented Design. (2021). Retrieved 04 Mayo 2021, from <https://gamesfromwithin.com/data-oriented-design>

Diseño a nivel de componentes. (2021). Retrieved 04 Mayo 2021, from https://virtual.itca.edu.sv/Mediadores/stis/37_diseo_a_nivel_de_componentes.html

Gitflow workflow. (2021). Retrieved 04 Mayo 2021, from <https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow>